

PARTE I - QUESTÕES DE MÚLTIPLA ESCOLHA

Questão 1 Resposta: c)

Questão 2 Resposta: b)

Questão 3 Resposta: c)

Questão 4 Resposta: b)

Questão 5 Resposta: c) **PARTE II - QUESTÕES DISCURSIVAS**

Questão 6

- **Teste automatizado:** É um script que verifica se o código funciona como esperado. Exemplo: um teste escrito em Jest que simula um alerta de enchente para garantir que a função de disparo não retorne erro.
- **Cobertura de testes:** Indica a porcentagem do código executada e validada pelos testes. Exemplo: O relatório mostra se a lógica que decide para qual região o alerta ambiental vai foi realmente executada pelo teste.
- **Vulnerabilidade de software:** É uma falha de segurança no código ou nas dependências. Exemplo: Uma biblioteca de terceiros no servidor Node.js que permite injeção de código, colocando em risco o sistema de comunicação de eventos climáticos do INPE.

Questão 7 O pipeline ter executado 300 testes com sucesso dá uma falsa sensação de segurança. Como a cobertura é de apenas 40%, os testes estão altamente concentrados em apenas algumas poucas áreas do sistema. É um risco enorme porque 60% do código (que pode envolver regras vitais de negócio ou lidar com estruturas de diretórios complexas) pode falhar em produção silenciosamente.

Questão 8 Eu recomendaria à equipe interromper o desenvolvimento de novas features e focar imediatamente em escrever testes para a camada de **Utils** (18%). Em projetos com Node e TypeScript, funções utilitárias costumam realizar conversões, formatações e cálculos (como dados geolocalizados do INPE). Se os **Utils** falharem silenciosamente por falta de testes, injetarão dados incorretos nos **Services** (90%) e **Controllers** (95%), causando falha em cascata.

Questão 9

Um sistema pode ter alta cobertura porque a cobertura mede apenas se a linha de código foi percorrida pela execução do teste, não se o comportamento do sistema está logicamente correto. Por exemplo, você pode rodar uma função de envio de dados de satélite do INPE num teste sem aplicar as *assertions* (validações) corretas. O código foi coberto, mas a lógica matemática de georreferenciamento poderia estar gerando resultados errados.

Questão 10

- **O que faz:** O `npm test` executa a suíte de testes configurada no projeto (como Jest ou Vitest).
- **Quando é executado:** Deve ser executado localmente antes do commit e, obrigatoriamente, automatizado pelo pipeline de CI/CD (geralmente via GitHub Actions rodando num ambiente Ubuntu) a cada push ou Pull Request.
- **Sua importância:** Ele atua como o "porteiro" do projeto INPE. Se o `npm test` falhar, o pipeline quebra, bloqueando o deploy de um código defeituoso que prejudicaria o monitoramento de eventos críticos.

Questão 11 Não concordo. Vulnerabilidades classificadas como *High* geralmente abrem margem para ataques como DoS (Denial of Service) ou execução de código remoto. Para um projeto governamental e crítico do INPE, expor o sistema a ataques pode tirar o aplicativo do ar no exato momento em que a população mais precisar relatar um evento crítico ambiental.

Questão 12 Vulnerabilidade em dependências de software ocorre quando utilizamos bibliotecas de terceiros que possuem falhas publicadas. Como em projetos JS a pasta `node_modules` é enorme, é comum arrastar pacotes antigos. Um exemplo no aplicativo do INPE seria usar uma biblioteca antiga de formatação de datas (como uma versão obsoleta do `moment`) que possui uma falha de "ReDoS" e permite que um usuário mal-intencionado trave o servidor backend enviando datas complexas para o sistema de monitoramento.

Questão 13 Não garante. Um badge "Coverage: 97%" indica um esforço excelente de automação, mas a qualidade técnica dos testes ainda precisa ser avaliada. Testes "viciados" podem passar pelas rotinas sem validar cenários negativos (edge cases) ou erros de integração de APIs, mantendo defeitos de lógica.

Questão 14

O relatório de cobertura dá ao gerente de projetos visibilidade técnica e embasamento em dados. Se uma nova *feature* for entregue pelos desenvolvedores e derrubar a cobertura geral, o gerente pode barrar a entrega exigindo que os testes faltantes sejam codificados. Ele reduz o "achismo" sobre a segurança do deploy.

Questão 15 A recomendação técnica é barrar a implantação imediatamente. Mesmo com "Tests: 100% Passed" e "Coverage: 86%", possuir 5 vulnerabilidades de nível *High* é inaceitável para segurança da informação. A equipe deve rodar `npm audit fix` ou atualizar as bibliotecas afetadas. O deploy só deve acontecer quando as vulnerabilidades críticas e altas forem resolvidas.

Questão 16 Integração Contínua (CI) e testes automatizados caminham juntos. A premissa do CI é conseguir integrar código constantemente sem quebrar o sistema. Isso é impossível sem testes automatizados rápidos e confiáveis que rodam automaticamente no repositório validando se as alterações recém-enviadas quebraram as regras de negócio pré-existent.

Questão 17 Essa funcionalidade (denúncias de queimadas da população) é vitalícia e de emergência. Exige uma estratégia rigorosa porque o aplicativo não pode perder um *payload* por erro de roteamento ou de banco de dados. Uma denúncia que falha em ser registrada

não é só um erro no log, significa falta de atendimento das autoridades que pode resultar em perdas severas.

Questão 18

- **Build:** Mostra se a aplicação compilou ou transpilou (ex: de Typescript para JS) sem erros de sintaxe ou de linkagem.
- **Tests:** Mostra visualmente se a última execução da esteira validou todos os testes (Passed) ou falhou.
- **Coverage:** Representa a métrica quantitativa (porcentagem) de validação de testes perante a base de código.
- **Vulnerabilities:** Reflete as varreduras de segurança nas dependências informando se há riscos de segurança reportados.

Questão 19 A afirmação de que "se a cobertura estiver acima de 90%, não precisamos mais revisar o código" está totalmente equivocada. Como falei anteriormente, a cobertura mede a execução de linha, não a qualidade da solução. O Code Review verifica boas práticas, legibilidade, design patterns, clareza das variáveis e a performance do código, coisas que um simples teste unitário passando muitas vezes ignora.

Questão 20 Antes de aprovar o deploy do INPE, eu analisaria:

1. **Pipeline Status:** O pipeline inteiro precisa estar verde (Build passou, os testes unitários e de integração passaram).
2. **Coverage Drop:** A cobertura não precisa ser 100%, mas o indicador mais importante é verificar se a cobertura *caiu* nesta nova PR. Se caiu, código sem teste foi submetido.
3. **Vulnerabilities Reports:** Garantir que o comando `npm audit` resultou em zero vulnerabilidades de nível Crítico ou *High*. Implantação de sistema governamental deve ter blindagem máxima nas dependências.